Table of Contents

StdfIgxlRecords.2.25

StdfIgxlRecords.2.26

STDFForIgxlDefTopic

IG-XL Support for STDF

IG-XL Support for STDF

This section contains the following topics:

- Overview of IG-XL and STDF
- General Topics

The section IG-XL Support for STDF Record Types contains the record-specific descriptions.

StdfIgxlIntro.1.2

<	>

IG-XL Support for STDF : Overview of IG-XL and STDF

Overview of IG-XL and STDF

Note that IG-XL supports the following specific version of STDF:

Version 4, December 1986

Other STDF versions are also labeled as Version 4. IG-XL supports the Version 4 that is dated December 1986.

The STDF specification itself is implementation-independent. Even though it was developed by Teradyne, however, IG-XL does not implement all of the specification, and it implements some of it in ways that differ from other companies and even from other Teradyne products.

This document describes how IG-XL supports STDF. It describes what records are supported and what is written to each field. It also describes the user interfaces that let you control what is written to the fields, with links to the documentation for those interfaces.

The section IG-XL Support for STDF Record Types contains the record-specific descriptions.

The two interfaces for user control over how STDF data is written are:

- DataCollect Setup, a GUI that you can use to set up datalogging before running the test program. See About DataCollect Setup.

- VBT, whose Datalog object gives programmatic control over datalog setup. See Datalog in the VBT Syntax Reference.

The IG-XL contains the STDF Specification and the ATDF Specification, an ASCII version of the STDF Specification:

- About STDF

- About ATDF

For the IG-XL utilities that can convert STDF files into readable format, see STDF Utilities in the DataCollect Setup section.

StdfIgxlIntro.1.3

< >

IG-XL Support for STDF : General Topics

General Topics

This section contains topics that apply to more than one record type:

- UNIX Time Format and Time Zones
- Extended Site Numbers and Counts

< >

StdfIgxlIntro.1.4

< >

IG-XL Support for STDF : General Topics : UNIX Time Format and Time Zones

UNIX Time Format and Time Zones

Times in STDF records are written in UNIX time format, which is a system for describing points in time: it is the number of seconds elapsed since midnight Coordinated Universal Time (UTC) of January 1, 1970, not counting leap seconds.

Time zones are not logged. Local time is always assumed. If you change the system's time zone on the fly, you must reload the program to display the correct local time.

< >

StdfIgxIntro.1.5

< >

IG-XL Support for STDF : General Topics : Extended Site Numbers and Counts

Extended Site Numbers and Counts

For STDF record types that contain a site number, the STDF specification defines the SITE_NUM field as a one-byte integer. The maximum value it can store is therefore 256, so it can represent sites 0-255. Because IG-XL can support more than 256 sites, it must use additional ways to represent the higher site numbers and site counts. It uses the HEAD_NUM field to support the extended site numbers. In addition, it writes a special ATR and GDR to the STDF file to indicate that extended site numbers are being used.

Formula for Site Number

In extended site numbering, the HEAD_NUM indicates a group of 256 sites and the SITE_NUM indicates the site within the group. To derive the actual site number, use this formula:

HEAD_NUM*256 + SITE_NUM

For sites 0-255, HEAD_NUM is 0, so SITE_NUM gives the site number. For site 256, HEAD_NUM is 1 and SITE_NUM is 0.

The ReadStdf output for a test program with 385 sites has one Part Count Record (PCR) with these fields:

Pcr.head_num = 1

Pcr.site_num = 107

The formula for reading this site number is as follows:

1*256 + 107 = 363

This PCR is for site 363.

Summary Records

Note that a HEAD_NUM value of 255 has a special meaning, whether or not extended site numbering is being used. It indicates that the record is a summary of all sites, not site-specific. When HEAD_NUM is 255, the SITE_NUM field does not indicate a site number and is therefore set to 0 or 255, depending on the record type.

This should not cause a problem even with extended site numbering, because having 255*256 sites is not practical.

Non-Extended Site Numbering

If the test program has 256 or fewer sites, the site-specific records contains the default HEAD_NUM value as defined for the record type; this is 0 or 1, depending on the type. The SITE_NUM field contains the actual site number.

For some record types, the default HEAD_NUM value is 1, even if the site count is greater than 256. For this reason, you cannot assume that a HEAD_NUM of 1 means that the actual site number is 1*256+SITE_NUM. Instead, you must check for the presence of the ATR or GDR, as described below. If these records are present, extended site numbering is being used. If they are not present, a HEAD_NUM of 1 is simply the default value and SITE_NUM contains the actual site number.

Site Description Record (SDR)

When extended site numbering is used, the Site Description Record (SDR) presents a different case. It has a SITE_GRP field that is used in a way similar to HEAD_NUM. See Site Description Record (SDR).

Extended Site Numbering Indicators: ATR and GDR

If the site count is greater than 256, IG-XL writes two special records to indicate that extended site numbering is being used:

- It adds an Audit Trail Record (ATR) after the FAR and before the MIR. The ATR field CMD_LINE contains the string EXTENDEDSITES, followed by a space and the number of sites in the test program.

- It adds a Generic Data Record (GDR) after the final SDR. The GDR has two data fields, the ASCII string EXTENDEDSITES and the site count.

The ReadStdf output for a test program with 385 sites shows the following ATR and GDR:

```
            Audit Trail Record
Atr.rec_len  =          22   MSB = 0x00,  LSB = 0x16
Atr.rec_typ  = 0
Atr.rec_sub  = 20
Atr.mod_tim  = 0x49b77a74 == Wed Mar 11 09:46:44 2009
Atr.cmd_line = EXTENDEDSITES 385
            Generic Data Record
Gdr.rec_len =          20   MSB = 0x00,  LSB = 0x14
Gdr.rec_typ = 50
Gdr.rec_sub = 10
Gdr.fld_cnt =           2   MSB = 0x00,  LSB = 0x02
```

Gdr.gen_data_typ = 10: ASCII string
EXTENDEDSITES
Gdr.gen_data_typ = 2: Unsigned short 0x0181 == 385

An STDF analysis program can check for either of these records. If either record is present in the STDF file, and the record contains the string EXTENDEDSITES as described above, then the analysis program knows that the site count is greater than 256 and that it should use HEAD_NUM or SITE_GRP in addition to SITE_NUM to determine the site number in site-specific records. If the records are not present, the site count is 256 or less, and the site-related fields have their default meanings.

StdfIgxlRecords.2.01

< >

IG-XL Support for STDF Record Types

IG-XL Support for STDF Record Types

This section explains how IG-XL writes the fields of the STDF records.

In this section, the STDF record types are given in the same order as in the STDF Specification, which is by the record type and sub-type codes. The following table lists the STDF record types in alphabetical order.

STDF Record Types

Abbreviation	Record Type
ATR	Audit Trail Record (ATR)
BPS	Begin Program Section Record (BPS)
DTR	Datalog Text Record (DTR)
EPS	End Program Section Record (EPS)
FAR	File Attributes Record (FAR)
FTR	Functional Test Record (FTR)
GDR	Generic Data Record (GDR)
HBR	Hardware Bin Record (HBR)
MIR	Master Information Record (MIR)
MPR	Master Information Record (MIR)
MRR	Master Results Record (MRR)
PCR	Part Count Record (PCR)

PGR	Pin Group Record (PGR)
PIR	Part Information Record (PIR)
PLR	Pin List Record (PLR)
PMR	Pin Map Record (PMR)
PRR	Part Results Record (PRR)
PTR	Parametric Test Record (PTR)
RDR	Retest Data Record (RDR)
SBR	Software Bin Record (SBR)
SDR	Site Description Record (SDR)
TSR	Test Synopsis Record (TSR)
WCR	Wafer Configuration Record (WCR)
WIR	Wafer Information Record (WIR)
WRR	Wafer Results Record (WRR)

In listing the fields of each record type, the following descriptions do not include the initial REC_LEN, REC_TYP, and REC_SUB fields. These values are defined in the STDF Specification for each record type, and IG-XL writes them according to the specification.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.02

< >

IG-XL Support for STDF Record Types : File Attributes Record (FAR)

File Attributes Record (FAR)
STDF Specification: File Attributes Record (FAR)
Contains the information necessary to determine how to decode the STDF data contained in the file.

FAR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
CPU_TYPE	U*1	CPU type that wrote this file	2 (According to the STDF Specification, 2 = Sun 386i computers, and IBM PC, IBM PC-AT, and IBM PC-XT computers, plus any other computers with compatible data formats.)
STDF_VER	U*1	STDF version number	4

< >

StdfIgxlRecords.2.03

< >

IG-XL Support for STDF Record Types : Audit Trail Record (ATR)

Audit Trail Record (ATR)
STDF Specification: Audit Trail Record (ATR)
Contains the information necessary to determine how to decode the STDF data contained in the file.

FAR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
MOD_TIM	U*4	Date and time of STDF file modification	See UNIX Time Format and Time Zones.
CMD_LINE	C*n	Command line of program	See note.

IG-XL writes an ATR to the STDF file only if the site count is greater than 255. In this case, the CMD_LINE field has the value EXTENDEDSITES followed by a space and the actual site count. If the site count is 255 or less, IG-XL does not write an ATR.
For more information, see Extended Site Numbers and Counts.

< >

StdfIgxlRecords.2.04

< >

IG-XL Support for STDF Record Types : Master Information Record (MIR)

Master Information Record (MIR)
STDF Reference: Master Information Record (MIR)
The MIR contains all the global information for a tested lot of parts that is available before testing begins. The Master Results Record (MRR) contains the global information that is available only after testing.
Each STDF file contains exactly one MIR.
Some MIR fields are written by IG-XL. These are shown below.
You can overwrite some fields that are written by IG-XL. These are explicitly noted below. In DataCollect Setup, the Lot Definition tab has fields that let you specify values to overwrite default values that IG-XL writes. In VBT, you use properties under TheExec.Datalog.Setup.LotSetup.
The rest of the fields are not written by IG-XL but can be written by the user. These are indicated below as "User-supplied." By default, IG-XL sets these to the value defined in the STDF Specification as indicating missing or invalid data. You can supply values for these fields using either DataCollect Setup or VBT syntax.
In DataCollect Setup, the Lot Definition tab has a More button that brings up the More Information dialog box. In this window, the MIR tab lets you supply values for the MIR fields that IG-XL does not write. Each field has a label and a text box, with a check box to the right. To write to an STDF field, enter the value you want. Check the box to have the value written to the text file and window outputs as well.
In VBT, you use properties under TheExec.Datalog.Setup.LotSetup. These properties are paired. One property has the field name and the other, a Boolean, has the field name prefaced with a "b": for example, .ProcessID and .bProcessID. To write to an STDF field, set the first property to the value you want. To have the value written to the text and window outputs as well, set the second property, with the "b," to True:

TheExec.Datalog.Setup.LotSetup.ProcessID = "BRN18"

TheExec.Datalog.Setup.LotSetup.bProcessID = True

TheExec.Datalog.ApplySetup

For each field labelled “User-supplied,” the following table gives the name of the field on the MIR tab in DataCollect Setup and the name of the VBT property under TheExec.Datalog.Setup.LotSetup.

MIR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
SETUP_T	U*4	Date and time of job setup	Date and time of the job setup. This corresponds to the first time a setup procedure is called during program validation. See UNIX Time Format and Time Zones.
START_T	U*4	Date and time first part tested	Date and time of the first part tested. See UNIX Time Format and Time Zones.
STAT_NUM	U*1		1
MODE_COD	C*1	Test mode code (e.g. prod, dev)	E in engineering mode and P in production mode. Can be overwritten by the user. See MODE_COD Values.
RTST_COD	C*1	Lot retest code	User-supplied. MIR tab: Retest Code (drop-down list includes numbers 0 through 9 plus Y and N and blank) VBT: RetestCode
PROT_COD	C*1	Data protection code	User-supplied. MIR tab: Prot Code (drop-down list includes numbers 0 through 9, letters A through Z, and blank) VBT: ProtCode
BURN_TIM	U*2	Burn-in time (in minutes)	65,535 (= invalid/missing data). User-supplied. MIR tab: Burn Time VBT: BurnTime
CMOD_COD	C*1	Command mode code	User-supplied.

			MIR tab: Command (drop-down list includes numbers 0 through 9 and letters A through Z) VBT: Command
LOT_ID	C*n	Lot ID	User-supplied. The Lot ID box is on the Lot Definition tab, not the MIR tab. VBT: LotID
PART_TYP	C*n	Part Type (or product ID)	User-supplied. The Part Type box is on the Lot Definition tab, not the MIR tab. VBT: PartType
NODE_NAM	C*n	Name of node that generated data	The node name of the tester or computer running IG-XL. This corresponds to the NetBIOS name of the local computer.
TSTR_TYP	C*n	Tester type	For FLEX and MicroFLEX, “Integra Flex.” For UltraFLEX, “Jaguar.”
JOB_NAM	C*n	Job name (test program name)	The currently active job name in DataTool. If there is no job name, then the program workbook name. You can specify that you want both the job name and the program name in this format: ProgramName.xls@jobname In DataCollect Setup: Setups tab, Display Options, Extend MIR Job Name In VBT: TheExec.Datalog.Setup.DatalogSetup.ExtendMirJobName
JOB_REV	C*n	Job revision number	User-supplied. MIR tab: Job Rev VBT: JobRev
SBLOT_ID	C*n	Sublot ID	User-supplied. MIR tab: Sub Lot VBT: SubLot
OPER_NAM	C*n	Operator name or ID (at setup time)	User’s log-on user name. Can be overwritten by the user: DataCollect Setup: the Operator field on the Lot Definition tab (not the MIR tab) VBT: Operator

EXEC_TYP	C*n	Tester executive software type	"IG-XL". Can be overwritten by the user: MIR tab: Exec Type VBT: ExecType
EXEC_VER	C*n	Tester exec software version number	Installed version of IG-XL. Can be overwritten by the user: MIR tab: Exec Ver VBT: ExecVer
TEST_COD	C*n	Test phase or step code	User-supplied. MIR tab: Test Code VBT: TestCode
TST_TEMP	C*n	Test temperature	User-supplied. MIR tab: Test Temp VBT: TstTemp
USER_TXT	C*n	Generic user text	User-supplied. MIR tab: User Text VBT: UserText
AUX_FILE	C*n	Name of auxiliary data file	User-supplied. MIR tab: Aux File VBT: AuxFile
PKG_TYP	C*n	Package type	User-supplied. MIR tab: PKG Type VBT: PkgType
FAMLY_ID	C*n	Product family ID	User-supplied. MIR tab: Family ID VBT: FamilyID
DATE_COD	C*n	Date code	User-supplied. MIR tab: Date Code VBT: DateCode
FACIL_ID	C*n	Test facility ID	User-supplied. MIR tab: Facility ID VBT: FacilityID

FLOOR_ID	C*n	Test floor ID	User-supplied. MIR tab: Floor ID VBT: FloorID
PROC_ID	C*n	Fabrication process ID	User-supplied. MIR tab: Process ID VBT: ProcessID
OPER_FRQ	C*n	Operation frequency or step	User-supplied. MIR tab: Oper Frq VBT: OperFrq
SPEC_NAM	C*n	Test spec name	User-supplied. MIR tab: Spec Name VBT: SpecName
SPEC_VER	C*n	Test spec version number	User-supplied. MIR tab: Spec Ver VBT: SpecVer
FLOW_ID	C*n	Test flow ID	User-supplied. MIR tab: Flow ID VBT: FlowID
SETUP_ID	C*n	Test setup ID	User-supplied. MIR tab: Setup ID VBT: SetupID
DSGN_REV	C*n	Device design revision	User-supplied. MIR tab: Design Rev VBT: DesignRev
ENG_ID	C*n	Engineering lot ID	User-supplied. MIR tab: Eng ID VBT: EngID
ROM_COD	C*n	ROM code ID	User-supplied. MIR tab: ROM ID VBT: RomID
SERL_NUM	C*n	Tester serial number	User-supplied. MIR tab: Serial Num VBT: SerialNum

SUPR_NAM	C*n	Supervisor name or ID	User-supplied. MIR tab: Supervisor VBT: Supervisor

MODE_COD Values

The MODE_COD field indicates the tester mode under which the lot was tested. The STDF Specification defines a set of alphabetic characters for specific modes, as follows:

A	AEL (Automatic Edge Lock) mode
C	Checker mode
D	Development/Debug test mode
E	Engineering mode (same as Development mode)
M	Maintenance mode
P	Production test mode
Q	Quality Control

All other alphabetic codes are reserved for future use by Teradyne. Characters 0 - 9 are available for customer use.

By default, IG-XL writes E in engineering mode and P in production mode.

You can overwrite this value using either DataCollect or VBT.

In DataCollect, the Lot Definition tab has the Test Mode control. This drop-down list contains the predefined list of modes, plus the custom numbers 0 through 9 and the value None.

In VBT, the property TheExec.Datalog.Setup.LotSetup.TestMode can set this field using integer values. See TestMode in the VBT Syntax Reference.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.05

< >

IG-XL Support for STDF Record Types : Master Results Record (MRR)

Master Results Record (MRR)
STDF Specification: Master Results Record (MRR)
The Master Results Record (MRR) contains lot information that is not available until the end of testing.
Each STDF file contains exactly one MRR.

MRR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
FINISH_T	U*4	Date and time last part tested	Date and time that the last part was tested. This corresponds to the time when the operator interface or VBT issues an EndLot, or when the Get Final Summary/End Lot button in the DataCollect Setup window is clicked. See UNIX Time Format and Time Zones.
DISP_COD	C*1		Not written.
USR_DESC	C*n	Lot description supplied by user	See USR_DESC and EXC_DESC.
EXC_DESC	C*n	Lot description supplied by exec	See USR_DESC and EXC_DESC.

USR_DESC and EXC_DESC

By default, IG-XL does not write valid values to these fields. You can specify a value to be written to these fields using:

Call TheExec.Datalog.Setup.SetMRRInfo(bstrUsrDesc, bstrExecDesc)

For more information, see SetMRRInfo in the VBT Syntax Reference.

DataCollect Setup has no control to specify these values.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.06

< >

IG-XL Support for STDF Record Types : Part Count Record (PCR)

Part Count Record (PCR)
STDF Specification: Part Count Record (PCR)
Contains the part count totals for one or all test sites.
IG-XL writes one PCR for each site to indicate the number of parts tests at that site.
In addition, IG-XL writes one PCR as a summary of all the sites. A HEAD_NUM of 255 indicates a summary PCR; the site number is set to 0.

PCR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	For a site-specific record, 1. To indicate a summary PCR, IG-XL sets this to 255.
SITE_NUM	U*1	Test site number	For a site-specific record, the site number. To indicate a summary PCR, IG-XL sets this to 0.
PART_CNT	U*4	Number of parts tested	Number of devices tested. The value 4,294,967,295 indicates missing/invalid data.
RTST_CNT	U*4	Number of parts retested	Number of passed devices retested. RTST_CNT is incremented if the PRR.PART_FLG bit 0 or bit 1 is set to 1.

			The value 4,294,967,295 indicates missing/invalid data.
ABRT_CNT	U*4	Number of aborts during testing	If TheExec.Datalog method WriteFunctionalResult, WriteParametricResult, or WriteParametricResultOptLoHi has its testFlag parameter set to logTestAbort, IG-XL calculates and writes the abort count at each site for the site-specific PCRs and the total abort count for the summary PCR. The value 4,294,967,295 indicates missing/invalid data.
GOOD_CNT	U*4	Number of good (passed) parts tested	Number of passed devices tested. The value 4,294,967,295 indicates missing/invalid data.
FUNC_CNT	U*4		Not written.

StdfIgxlRecords.2.07

<	>

IG-XL Support for STDF Record Types : Hardware Bin Record (HBR)

Hardware Bin Record (HBR)
STDF Specification: Hardware Bin Record (HBR)
Stores a count of the parts “physically” placed in a particular bin after testing.

HBR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	For a site-specific record: If sites<=256, 0. If sites>256, see Extended Site Numbering. To indicate a site-summary HBR, IG-XL sets this to 255.
SITE_NUM	U*1	Test site number	For a site-specific record: If sites<=256, the site number. If sites>256, see Extended Site Numbering. To indicate a site-summary HBR, IG-XL sets this to 255.
HBIN_NUM	U*2	Hardware bin number	The hardware bin number. For an HBR created to correspond to an SBR, IG-XL writes the number 65535.
HBIN_CNT	U*4	Number of parts in bin	The number of parts assigned to this bin.

HBIN_PF	C*1	Pass/fail indication	Default = blank See Bin Name and Pass/Fail Result.
HBIN_NAM	C*n	Name of hardware bin	Default = blank See Bin Name and Pass/Fail Result.

Number of Site-Specific HBRs

IG-XL writes one HBR per hardware bin for each site: that is, number of parts tested at that site that have been assigned to that bin.

IG-XL writes an HBR for every SBR, even if no hardware bin number is specified. That is, if a Flow Table or Bin Table step assigns a sort number but not a bin number, IG-XL writes an SBR but also creates a corresponding HBR with the bin number 65535. (In the datalog output window, such bins appear with the bin number "N/A.")

If no bins are specified, IG-XL writes one HBR and one SBR, both with the number 65535.

Summary HBRs

In addition to site-specific HBRs, if summary by site is requested, a summary HBR is written for each bin, showing how many parts tested at all sites were assigned to the bin. To request summary by site, use one of these methods:

- DataCollect Setup: on the Summary Report tab, check Total Summary (By Site).

- In VBT: TheExec.Datalog.Setup.SummarySetup.SiteReport = True

A HEAD_NUM value of 255 indicates a TSRs. See Test Synopsis Record (TSR).

Bin Name and Pass/Fail Result

If you want values for the bin name and pass/fail result, call TheExec.Datalog.HBRFill(HB_NUM, bstrHBName). It writes the name you specify to the HBIN_NAM field. It also writes P or F to HBIN_PF, based on the result specified in the Flow Table or Bin Table step that assigns parts to this bin. To have the pass/fail result filled in, use HBRFill to specify a bin name. See TheExec.Datalog.HBRFill in the VBT Syntax Reference.

Extended Site Numbering

For site-specific records, if the site count is greater than 256, extended site numbering is used. Extended site numbering makes special use of the HEAD_NUM and SITE_NUM fields. HEAD_NUM indicates a group of 256 sites and SITE_NUM indicates the site within the group. To derive the actual site number, use this formula: HEAD_NUM*256 + SITE_NUM.

For more information, see Extended Site Numbers and Counts.

StdfIgxlRecords.2.08

< >

IG-XL Support for STDF Record Types : Software Bin Record (SBR)

Software Bin Record (SBR)
STDF Specification: Software Bin Record (SBR)
Stores a count of the parts associated with a particular logical bin after testing.

SBR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	For a site-specific record, 0. To indicate a site-summary SBR, IG-XL sets this to 255.
SITE_NUM	U*1	Test site number	For a site-specific record, the site number. To indicate a site-summary SBR, IG-XL sets this to 0.
SBIN_NUM	U*2	Software bin number	The software bin number. For an SBR created to correspond to an HBR, IG-XL writes the number 65535.
SHBIN_CNT	U*4	Number of parts in bin	The number of parts assigned to this bin.
SBIN_PF	C*1	Pass/fail indication	Default = blank See “Bin Name and Pass/Fail Result” below.
HBIN_NAM	C*n	Name of software bin	Default = blank See “Bin Name and Pass/Fail Result” below.

Number of Site-Specific SBRs

IG-XL writes one SBR per software bin for each site: that is, number of parts tested at that site that have been assigned to that bin.

IG-XL writes an SBR for every HBR, even if no software bin number is specified. That is, if a Flow Table or Bin Table step assigns a bin number but not a sort number, IG-XL writes an HBR but also creates a corresponding SBR with the bin number 65535. (In the datalog output window, such bins appear with the bin number "N/A.")

If no bins are specified, IG-XL writes one HBR and one SBR, both with the number 65535.

Summary SBRs

In addition to the site-specific SBRs, if summary by site is requested, a summary SBR is written for each bin, showing how many parts tested at all sites were assigned to the bin. To request summary by site, use one of these methods:

- DataCollect Setup: on the Summary Report tab, check Total Summary (By Site).
- In VBT: TheExec.Datalog.Setup.SummarySetup.SiteReport = True

A HEAD_NUM value of 255 indicates a site-summary SBR; the site number is set to 0.

Bin Name and Pass/Fail Result

If you want values for the bin name and pass/fail result, call TheExec.Datalog.SBRFill(SB_NUM, bstrSBName). It writes the name you specify to the SBIN_NAM field. It also writes P or F to SBIN_PF, based on the result specified in the Flow Table or Bin Table step that assigns parts to this bin. To have the pass/fail result filled in, use SBRFill to specify a bin name. See TheExec.Datalog.SBRFill in the VBT Syntax Reference.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.09

< >

IG-XL Support for STDF Record Types : Pin Map Record (PMR)

Pin Map Record (PMR)
STDF Specification: Pin Map Record (PMR)
Provides indexing of pin names and maps them to sites and associated information. Each PMR contains the information for a single channel/pin combination.

PMR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
PMR_INDX	U*2	Unique index associated with pin	Starts with 1 for the first pin and increments by 1 for each subsequent pin in the Pin Map sheet. Other record types use this number to refer to this specific pin.
CHAN_TYP	U*2	Channel type	See Channel Type Values.
CHAN_NAM	C*n	Channel name	Channel name, in Pogo format, as given in the Siten column of the Channel Map sheet row for this pin.
PHY_NAM	C*n	Physical name of pin	Value of the Package Pin column of the Channel Map sheet row for this pin. This is an optional value in the channel map.
LOG_NAM	C*n	Logical name of pin	Value of the Pin Name column of the Channel Map sheet row for this pin.

HEAD_NUM	U*1	Head number associated with channel	0.
SITE_NUM	U*1	Site number associated with channel	The site number for this pin/channel combination, as reflected in the n of the Siten column header.

Number of PMRs

Each pin and channel combination on a Channel Map sheet has one PMR. For example, a four-site channel map will have four PMRs for each pin. Each PMR will have the same PMR_INDX and LOG_NAM values, but different CHAN_NAM and SITE_NUM values.

If a pin is connected to multiple resources, either directly (syntax :M) or using DIB access (:DA), or if it is a combined connection pin (:C), the pin has multiple rows in the channel map. In these cases, only the first pin row is represented in the PMRs. Rows that have the :M, :DA, or :C syntax have no PMRs.

Channel Type Values

The CHAN_TYP field indicates the type of the channel to which the pin is connected. This is an integer field, so integers are used to represent channel types. This section describes how those integers are mapped to channel types.

J750 and FLEX use one set of values. These values are used for UltraFLEX as well, although it also has additional values. J750 and FLEX channel types are:

- 0=Input
- 1=Output
- 2=I/O
- 3=DPS
- 4=Gnd
- 5=N/C
- 6=Utility

- 7=Analog
- 8=CTSrc
- 9=CTCap
- 10=CTRefA
- 11=CTRefB
- 12=Power
- 13=Unk

On FLEX, all analog instruments are datalogged as 13, that is, as Unknown.
UltraFLEX uses these types:

- 0: chInput
- 1: chOutput
- 2: chIO
- 3: chDPS
- 4: chGnd
- 5: chNC
- 6: chUtil
- 7: chAnalog

- 8: chCTSrc
- 9: chCTCap
- 10: chCTRefA
- 11: chCTRefB
- 12: chPower
- 13: chUnk
- 30: chAWGNeg
- 31: chAWGPos
- 32: chBBACCapNeg
- 33: chBBACCapPos
- 34: chBBACSrcNeg
- 35: chBBACSrcPos
- 36: chBBACSrcRef
- 37: chDCDiffMeter
- 38: chDCIO
- 39: chDCIOMerged02
- 40: chDCIOMerged04

- 41: chDCIOMerged08
- 42: chDCIOMerged16
- 43: chDCIOMerged32
- 44: chDCTime
- 45: chDCTimeHP
- 46: chDCTimeTrig
- 47: chDCVI
- 48: chDCVIMerged
- 49: chDCVS
- 50: chDCVSMerged2
- 51: chDCVSMerged4
- 52: chDCVSMerged6
- 53: chDSPFake
- 54: chGigaDigNeg
- 55: chGigaDigPos
- 57: chMW
- 58: chMWSource

- 59: chPMO
- 60: chSerial
- 61: chTurboACCapNeg
- 62: chTurboACCapPos
- 63: chTurboACSrcNeg
- 64: chTurboACSrcPos
- 65: chTurboACSrcRef
- 67: chVHFACCapturePrimaryNeg
- 68: chVHFACCapturePrimaryPos
- 69: chVHFACCaptureSecondaryNeg
- 70: chVHFACCaptureSecondaryPos
- 71: chVHFACSerialBusClock
- 72: chVHFACSerialBusIO0
- 73: chVHFACSerialBusIO1
- 74: chVHFACSourceEvent0
- 75: chVHFACSourceEvent1
- 76: chVHFACSourceEvent2

- 77: chVHFACSourceEvent3
- 78: chVHFACSourceEvent4
- 79: chVHFACSourceEvent5
- 80: chVHFACSourceEvent6
- 81: chVHFACSourceEvent7
- 82: chVHFACSourcePrimaryNeg
- 83: chVHFACSourcePrimaryPos
- 84: chVHFACSourceSecondaryNeg
- 85: chVHFACSourceSecondaryPos
- 86: chVHFACTimePrimaryNeg
- 87: chVHFACTimePrimaryPos
- 88: chVHFACTimeSecondaryNeg
- 89: chVHFACTimeSecondaryPos
- 90: chVHFACTrigControl0
- 91: chVHFACTrigControl1
- 100: chDCVSMerged8
- 101: chBy16IO

- 102: chBy16Drive
- 103: chBy16DiffIO
- 104: chBy16DiffDrive
- 105: chBy16DiffClk
- 106: chHSD4GPPMU
- 107: chUltraCapture
- 108: chUltraSource

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.10

< >

IG-XL Support for STDF Record Types : Pin Group Record (PGR)

Pin Group Record (PGR)

STDF Specification: Pin Group Record (PGR)

Defines a pin group. Each pin group defined on a Pin Map sheet has one PGR.

Each PGR lists the pins in the pin group. If the Pin Map sheet pin group definition contains another pin group, the PGR lists the individual pins in the contained pin group.

Data Fields

PGR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
GRP_INDX	U*2	Unique index associated with pin group	32768 for first pin group. Increment by 1 for each subsequent pin group.
GRP_NAM	C*n	Name of pin group	Pin group name as given in Pin Map sheet.
INDX_CNT	U*2	Count (k) of PMR indexes	Number of pins in the pin group.
PMR_INDX	kxU*2	Array of indexes for pins in the group	An array of the PMR indexes of the pins in the pin group. To find the name of each pin, use this PMR_INDX number to match the PMR_INDX number on the PMR for this pin.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.11

< >

IG-XL Support for STDF Record Types : Pin List Record (PLR)

Pin List Record (PLR)
STDF Specification: Pin List Record (PLR)
Displays the programmed state and returned state characters.
Each STDF file contains exactly one PLR.

PLR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
GRP_CNT	U*2	Count (k) of pins or pin groups	Number of pin groups plus the number of pins not in pin groups.
GRP_INDX	kxU*2	Array of pin or pin group indexes	The index number from the PMR for each pin not in a pin group, followed by the index number from the PGR for each pin group.
GRP_MODE	kxU*2		Not written (array of 0s = missing data).
GRP_RADX	kxU*1		Not written (array of 0s = missing data).
PGM_CHAR	kxC*1	Program state characters	See Programmed and Returned States.
RTN_CHAR	kxC*1	Return state characters	See Programmed and Returned States.
PGM_CHAL	kxC*1		Not written (array of 0s = missing data).
RTN_CHAL	kxC*1		Not written (array of 0s = missing data).

Programmed and Returned States

PGM_CHAR and RTN_CHAR display valid possible programmed states as defined for each tester's digital instrument. The fields are arrays, with the number of array members determined by the value of GRP_CNT, that is, the number of pin groups plus the number of pins not in pin groups. All elements of each array are identical, because the valid programmed and returned values are the same for all pins.

The strings for PGM_CHAR are:

- FLEX: 01LHMVXW
- UltraFLEX: 01LHMVXW*

The strings for RTN_CHAR are:

- FLEX: LHMG
- UltraFLEX: LHMG*

StdfIgxlRecords.2.12

< >

IG-XL Support for STDF Record Types : Retest Data Record (RDR)

Retest Data Record (RDR)
STDF Specification: Retest Data Record (RDR)
Signals that the data in this STDF file is for retested parts.
Each STDF file contains at most one RDR. If the file contains the record, it must come immediately after the MIR.
By default, writing retest data is not enabled. You must explicitly enable it in the test program using TheExec.Datalog.Setup.DatalogSetup.EnableRetestDataRecord. You then specify the bins containing retest data using TheExec.Datalog.Setup.DatalogSetup.RetestBins.

PLR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
NUM_BINS	U*2	Number (k) of bins being retested	Count of the bins specified by .RetestBins.
RTST_BIN	kxU*2	Array of retest bin numbers	The bin numbers specified by .RetestBins.

If writing the RDR was not enabled when the STDF file was created, you can still identify a retested DUT. If the PRR.PART_FLG bit 0 or bit 1 is set to 1, this setting means that this is not a new part. This indicates a retested part. The PIR, PTR, MPR, FTR, and PRR records that make up the current sequence (identified as having the same HEAD_NUM and SITE_NUM) supersede any previous sequence of records for the same part.

- If bit 0 is set to 1, then the current sequence supersedes any previous sequence of records with the same PART_ID.

- If bit 0 is set to 1, then the current sequence supersedes any previous sequence of records with the same X_COORD and Y_COORD.

See Parametric Test Record (PTR).

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.13

<	>

IG-XL Support for STDF Record Types : Site Description Record (SDR)

Site Description Record (SDR)
STDF Specification: Site Description Record (SDR)
Contains the number of sites in the program, plus any hardware attached to the tester.
Some SDR fields are written by IG-XL. These are noted below.
Other fields are not written by IG-XL but can be written by the user. These are indicated below as "User-supplied." By default, IG-XL sets these to the value defined in the STDF Specification as indicating missing or invalid data. You can supply values for these fields using either DataCollect Setup or VBT syntax.
In DataCollect Setup, the Lot Definition tab has a More button that brings up the More Information dialog box. (The Lot Definition tab itself has some fields that let you specify values to overwrite default values that IG-XL writes.) In this window, the SDR tab lets you supply values for the SDR fields that IG-XL does not write. Each field has a label and a text box, with a check box to the right. To write to the STDF field, enter the value you want. Check the box to have the value written to the text file and window outputs as well.
In VBT, you use properties under TheExec.Datalog.Setup.SiteSetup. These properties are paired. One property has the field name and the other, a Boolean, has the field name prefaced with a "b": for example, .HandType and .bHandType. To write to the STDF field, set the first property to the value you want. To have the value written to the text and window outputs as well, set the second property, with the "b," to True:

```
TheExec.Datalog.Setup.SiteSetup.HandType = "BRN18"
TheExec.Datalog.Setup.SiteSetup.bHandType = True
TheExec.Datalog.ApplySetup
```

For each field labelled "User-supplied," the following table gives the name of the field on the SDR tab in DataCollect Setup and the name of the VBT property under TheExec.Data.Setup.SiteSetup.

SDR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	0.
SITE_GRP	U*1	Site group number	If sites<=256, 255 (indicates that site groups are not supported). If sites>256, see Extended Site Numbering for SDRs.
SITE_CNT	U*1	Number (k) of test sites in site group	Number of sites on the channel map. This is the number of existing sites, not the number of starting sites. If sites<=256, this is the actual number of sites. If sites>256, see Extended Site Numbering for SDRs.
SITE_NUM	kxU*1	Array of test site numbers	Array of the site numbers on the Channel Map sheet. If sites<=256, these are the actual site numbers. If sites>256, see Extended Site Numbering for SDRs.
HAND_TYP	C*n	Handler or prober type	User-supplied. SDR tab: Hand Type VBT: HandType
HAND_ID	C*n	Handler or prober ID	User-supplied. SDR tab: Hand ID VBT: HandID
CARD_TYP	C*n	Probe card type	User-supplied. SDR tab: Card Type VBT: CardType
CARD_ID	C*n	Probe card ID	User-supplied. SDR tab: Card ID VBT: CardID
LOAD_TYP	C*n	Load board type	User-supplied. SDR tab: Load Type

			VBT: LoadType
LOAD_ID	C*n	Load board ID	User-supplied. SDR tab: Load ID VBT: LoadID
DIB_TYP	C*n	DIB board type	IG-XL tries to read this value from hardware. If it succeeds, the field becomes read-only. If it fails, you can write to the field: SDR tab: Dib Type VBT: DibType
DIB_ID	C*n	DIB board ID	IG-XL tries to read this value from hardware. If it succeeds, the field becomes read-only. If it fails, you can write to the field: SDR tab: Dib ID VBT: DibID
CABL_TYP	C*n	Interface cable type	User-supplied. SDR tab: Cabl Type VBT: CablType
CABL_ID	C*n	Interface cable ID	User-supplied. SDR tab: Cabl ID VBT: CablID
CONT_TYP	C*n	Handler contactor type	User-supplied. SDR tab: Cont Type VBT: ContType
CONT_ID	C*n	Handler contactor ID	User-supplied. SDR tab: Cont ID VBT: ContID
LASR_TYP	C*n	Laser type	User-supplied. SDR tab: Lasr Type VBT: LasrType
LASR_ID	C*n	Laser ID	User-supplied. SDR tab: Lasr ID VBT: LasrID

EXTR_TYP	C*n	Extra equipment type field	User-supplied. SDR tab: Extr Type VBT: ExtrType
EXTR_ID	C*n	Extra equipment ID	User-supplied. SDR tab: Extr ID VBT: ExtrID

Extended Site Numbering for SDRs

On the SDR, SITE_CNT and SITE_NUM are one-byte integers. For this reason, they can support only 256 sites, with a maximum site number of 255. Because IG-XL can support more than 256 sites, the SDR also uses the SITE_GRP field to indicate higher site numbers.

If the site count is greater than 256, IG-XL uses extended site numbering. For more information, see Extended Site Numbers and Counts.

If extended site numbering is used, each SDR represents a maximum of 128 sites. There is one SDR for each group of 128 sites, plus one for the final group of fewer than 128 sites, if there are any. For example, if a test program has 386 sites, the STDF file contains 4 SDRs: 3 with 128 sites (3*128 = 384), plus a fourth for the final 2 sites. (Parts of this example are shown below.)

With extended site numbering, SDR fields have these meanings:

SITE_GRP	The number of this group of 128 sites, starting with 0 for the first SDR.
SITE_CNT	The number of sites included in this SDR. This is 128, except for the final SDR (unless the total number of sites is a multiple of 128).
SITE_NUM	An array with SITE_CNT number of elements. The array contains the site numbers, from 0 to SITE_CNT-1. For all but the final SDR, this is an array from 0 to 127. To determine an actual site number on an SDR, use 128*SITE_GRP + SITE_NUM.

For example, the ReadStdf output for a test program with 386 sites has these SDRs:

```
[ First SDR - sites 0-127] ...
Sdr.head_num = 0
Sdr.site_grp = 0
Sdr.site_cnt = 128
Sdr.site_num:
        0 =   0
        1 =   1
        ...
        127 = 127
```

Sdr.hand_typ = not present not valid

...

...

[Second SDR - sites 128-255] ...

Sdr.head_num = 0

Sdr.site_grp = 1

Sdr.site_cnt = 128

Sdr.site_num:

0 = 0

1 = 1

...

127 = 127

Sdr.hand_typ = not present not valid

...

[Third SDR - sites 256-383] ...

Sdr.head_num = 0

Sdr.site_grp = 2

Sdr.site_cnt = 128

Sdr.site_num:

0 = 0

1 = 1

...

127 = 127

Sdr.hand_typ = not present not valid

...

[Fourth SDR - sites 384-385] ...

Sdr.site_grp = 3

Sdr.site_cnt = 2

Sdr.site_num:

0 = 0

1 = 1

Sdr.hand_typ = not present not valid

...

StdfIgxlRecords.2.14

< >

IG-XL Support for STDF Record Types : Wafer Information Record (WIR)

Wafer Information Record (WIR)
STDF Specification: Wafer Information Record (WIR)
Indicates where testing of a wafer begins for each wafer tested.
One WIR is written for each wafer tested.

WIR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_GRP	U*1	Site group number	255 (indicates missing/invalid data; IG-XL does not support site groups)
START_T	U*4	Date and time first part tested	Date and time that the first part on the wafer is tested. This corresponds to the StartOfWafer event. See UNIX Time Format and Time Zones.
WAFER_ID	C*n	Wafer ID	The user-supplied string that identifies this wafer. Typically, the equipment driver raises the AssertStartOfWafer event at the beginning of each new wafer. The event retrieves the wafer ID information from the prober and writes the ID to this field, overwriting any value that is

already there. The ID is updated for each new wafer.
You can also use these methods:

- From DataCollect Setup, the Wafer Information tab has the Wafer ID field.

- From VBT, use TheExec.Datalog.Setup .WaferSetup.ID.

These methods create a static ID that would be written for all wafers.
The default is a space, indicating a missing or invalid value.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.15

<	>

IG-XL Support for STDF Record Types : Wafer Results Record (WRR)

Wafer Results Record (WRR)
STDF Specification: Wafer Results Record (WRR)
Contains the result information relating to each wafer tested.
One WRR is written for each wafer tested.
\

WRR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_GRP	U*1	Site group number	255 (indicates missing/invalid data; IG-XL does not support site groups).
FINISH_T	U*4	Date and time last part tested	Date and time when IG-XL recognizes that no more parts on the wafer are to be tested. This corresponds to the EndOfWafer event. See UNIX Time Format and Time Zones.
PART_CNT	U*4	Number of parts tested	Number of parts tested. This does not include retested parts.
RTST_CNT	U*4	Number of parts retested	Number of parts retested. Each tested unit is counted either in PART_CNT or RTST_CNT, depending on how the site was enabled, for regular test or

			retest. If a part is retested, it is counted under RTST_CNT, not under PART_CNT.
ABRT_CNT	U*4		4,294,967,295 (indicates missing/invalid data)
GOOD_CNT	U*4	Number of good (passed) parts tested	Number of parts passed during testing.
FUNC_CNT	U*4		4,294,967,295 (indicates missing/invalid data)
WAFER_ID	C*n	Wafer ID	The wafer ID that appears in the corresponding WIR. See Wafer Information Record (WIR).
FABWF_ID	C*n	Wafer ID	See User-Supplied Values.
FRAME_ID	C*n	Fab wafer ID	See User-Supplied Values.
MASK_ID	C*n	Wafer frame ID	See User-Supplied Values.
USR_DESC	C*n	Wafer mask ID	See User-Supplied Values.
EXC_DESC	C*n	Wafer description supplied by user	See User-Supplied Values.

User-Supplied Values

By default, IG-XL does not write valid values to the last five fields of the record. You can specify a value to be written to these fields using:

```
Call TheExec.Datalog.Setup.WaferSetup.SetWaferInfo(WaferFabId, _
        WaferFrameId, WaferMaskId, WaferUserDesc, WaferExecDesc)
```

For more information, see SetWaferInfo in the VBT Syntax Reference.

DataCollect Setup has no control to specify these values.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.16

< >

IG-XL Support for STDF Record Types : Wafer Configuration Record (WCR)

Wafer Configuration Record (WCR)
STDF Specification: Wafer Configuration Record (WCR)
Contains the configuration information for all wafers tested by the test program.
One WCR is written per STDF file.
All WCR values are user-supplied. If you do not supply a value, IG-XL writes the value indicating missing or invalid data.
In DataCollect Setup, use the Wafer Information tab to supply these. In VBT, use the properties and methods under TheExec.Datalog.Setup.WaferSetup. The table below shows the field name, the property or method, and the default invalid/missing data value.

WCR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
WAFR_SIZ	R*4	Diameter of wafer in WF_UNITS	Wafer Information tab: Diameter of Wafer VBT: Diameter Default: 0
DIE_HT	R*4	Height of die in WF_UNITS	Wafer Information tab: Height of Die VBT: DieHeight Default: 0
DIE_WID	R*4	Width of die in WF_UNITS	Wafer Information tab: Width of Die VBT: DieWidth Default: 0

WF_UNITS	U*1	Units for wafer and die dimensions	The units for the values given in WAFR_SIZ, DIE_HT, and DIE_WID. The valid integer values and their meanings are: • 0: Unknown units • 1: Inches • 2: Centimeters • 3: Millimeters • 4: Mils Wafer Information tab: Unit (drop-down list contains the four unit names, plus Unknown). VBT: Unit Default: 0 (= unknown)
WF_FLAT	C*1	Orientation of wafer flat	The valid one-character values and their meanings are: • U: Up • D: Down • L: Left • R: Right • space: Unknown Wafer Information tab: Orientation of Wafer Flat (drop-down list contains the four orientations, plus Unknown).

			VBT: Orient Default: space (= unknown)
CENTER_X	I*2	X coordinate of center die on wafer	Wafer Information tab: Coordinate of Center Die - X VBT: XCenter Default: 0
CENTER_Y	I*2	Y coordinate of center die on wafer	Wafer Information tab: Coordinate of Center Die - Y VBT: YCenter Default: 0
POS_X	C*1	Positive X direction of wafer	The valid one-character values and their meanings are: • L: Left • R: Right • space: Unknown Wafer Information tab: Direction of Wafer - X (drop-down list contains the values Unknown, Right, and Left) VBT: XPos Default: space (= unknown)
POS_Y	C*1	Positive Y direction of wafer	The valid one-character values and their meanings are: • U: Up • D: Down • space: Unknown Wafer Information tab: Direction of Wafer - Y (drop-down list contains the values Unknown, Up, and Down)

VBT: YPos

Default: space (= unknown)

< >

StdfIgxlRecords.2.17

< >

IG-XL Support for STDF Record Types : Part Information Record (PIR)

Part Information Record (PIR)
STDF Specification: Part Information Record (PIR)
Indicates where testing of a particular part begins for each part tested by the test program.

PIR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_NUM	U*1	Test site number	The site where the part is tested.

PIR/PRRs and Sampling
The number of PIR/PRR pairs written depends on whether sample datalogging is being used. If there is no sampling and every device is being datalogged, then a PIR/PRR pair is written for each device.
If sample datalogging is being used, for a sample rate of n, every nth device is datalogged. By default, a PIR/PRR pair for a device is written only if the device is datalogged.
You can ensure that a PIR/PRR pair is written for each device using one of these methods:

- In DataCollect Setup, on the Setups tab Display Options window, check the box labelled Full Sample STDF Part Records.

- In VBT, set this property to True:
  TheExec.Datalog.setup.DatalogSetup.FullSampleStdfPartRecords

If this option is enabled, the default behavior is overridden. A PIR/PRR pair is written for every device, whether or not the device is datalogged. In addition, a BPS/EPS pair is written for each test run, even if no devices are datalogged in the test run.

< >

StdfIgxlRecords.2.18

< >

IG-XL Support for STDF Record Types : Part Results Record (PRR)

Part Results Record (PRR)
STDF Specification: Part Results Record (PRR)
Contains the result information relating to each part tested by the test program.
For the number of PIR/PRR pairs in the STDF file, see PIR/PRRs and Sampling under the description of the PIR.

PRR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_NUM	U*1	Test site number	The site number where the part was tested.
PART_FLG	B*1	Part information flag	The field, taken as a whole, has this meaning: • 0x00, if a new part passes • 0x01 or 0x02, if a retested part passes • 0x08, if a new part fails • 0x09 or 0x0A, if a retested part fails

			According to the STDF Specification, the meaning of the bits is as follows: • bit0 = bit1 = 0: The part was tested for the first time (not a retest). • bit0 XOR bit1 = 1: This is a retested part (on bit 0 vs. bit 1, see Retest Data Record (RDR)). • bit2: Test completion was normal (=0) or abnormal (=1). • bit3: The part passed (=0) or failed (=1). • bit4: The result (represented by bit3) is valid (=0) or invalid (=1). • bit5..bit7: reserved, must be 0. If the part passes, 0x00. Otherwise, 0x08.
NUM_TEST	U*2	Number of tests executed	The number of tests executed on this part at this site.
HARD_BIN	U*2	Hardware bin number	The number of the hardware bit (Bin columns) to which the part is assigned. If no Bin number is assigned, 65535 is written (= missing/invalid data).
SOFT_BIN	U*2	Software bin number	The number of the software bit (Sort columns) to which the part is assigned. If no Sort number is assigned, 65535 is written (= missing/invalid data).
X_COORD	I*2	(Wafer) X coordinate	-32768 (= missing/invalid data). The prober should set this value correctly.

Y_COORD	I*2	(Wafer) Y coordinate	-32768 (= missing/invalid data). The prober should set this value correctly.
TEST_T	U*4	Elapsed test time in milliseconds	The test time in milliseconds.
PART_ID	C*n	Part identification	The part identification number. By default, the starting number is 1 and is automatically incremented for each subsequent part. To specify a nondefault starting part number: • In DataCollect Setup, on the Lot Definition tab, use the Device Number field. • In VBT, use TheExec.Datalog.Setup.LotSetup .DeviceNumber
PART_TXT	C*n	Part description text	See PART_TXT and PART_FIX.
PART_FIX	B*n	Part repair information	See PART_TXT and PART_FIX.

PART_TXT and PART_FIX

By default, IG-XL does not write valid values to these fields. You can specify a value to be written to these fields using:

```
Call TheExec.Datalog.Setup.SetPRRPartInfo(selectionFlag, SiteNumber, _
                                          partfix(), parttxt)
```

For more information, see SetPRRPartInfo in the VBT Syntax Reference. In particular, note the description on how to create the array for the PART_FIX value.

DataCollect Setup has no control to specify these values.

StdfIgxlRecords.2.19

<	>

IG-XL Support for STDF Record Types : Test Synopsis Record (TSR)

Test Synopsis Record (TSR)
STDF References: Test Synopsis Record (TSR)
Contains a synopsis of the test execution and failure counts for a test.
IG-XL always writes summary TSRs, which are a summary of the test execution at all sites. Each test executed has one such summary TSR. A summary TSR has the value 255 in the HEAD_NUM and SITE_NUM fields.
In addition, IG-XL can write site-specific TSRs, which report on the execution of a test at a specific site. A site-specific TSR has the site number in the SITE_NUM field. To request site-specific TSRs, use one of these methods:

- DataCollect Setup: on the Summary Report tab, check Total Summary (By Site).
- In VBT: TheExec.Datalog.Setup.SummarySetup.SiteReport = True

Note that these controls also affect HBRs and SBRs. See Summary HBRs and Summary SBRs.

TSR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
HEAD_NUM	U*1	Test head number	For a site-specific TSR, 1. To indicate a summary of all sites, IG-XL sets this to 255.
SITE_NUM	U*1	Test site number	For a site-specific TSR, the site number. To indicate a summary of all sites, IG-XL sets

			this to 255.
TEST_TYP	C*1	Test type	• P: Parametric test • M: Multiple-result parametric test • F: Functional test
TEST_NUM	U*4	Test number	The TNum number for this test in the Flow Table sheet. If TNum is not specified, increment the previous test number by 1. If no TNum values are specified, the first test starts at 0.
EXEC_CNT	U*4	Number of test executions	Number of times this test was executed.
FAIL_CNT	U*4	Number of test failures	Number of times this test failed.
ALRM_CNT	U*4	Number of alarmed tests	Number of times this test was alarmed.
TEST_NAM	C*n	Test name	See Test Name.
SEQ_NAME	C*n	Sequencer (flow) name	See “Sequencer” Name.
TEST_LBL	C*n	Test label or text	See TEST_LBL.
OPT_FLAG	B*1	Optional data flag	Contains the following fields: • bit 0 set: TEST_MIN value is invalid • bit 1 set: TEST_MAX value is invalid • bit 2 set: TEST_TIM value is invalid • bit 3: Reserved for future use and must be 1

			• bit 4 set: TST_SUMS value is invalid • bit 5 set: TST_SQRS value is invalid • bits 6 - 7: Reserved for future use and must be 1 See TSR Statistics. If TSR statistics are not enabled (default), then the fields following this field are invalid, and this field is set to 0xff to indicate that the fields are invalid. If TSR statistics are enabled, then bits 0, 1, 2, 4, and 5 are not set to indicate that these fields are valid. In addition, bits 3, 6, and 7 are reserved and must be set to 1. The resulting value is therefore 0xc8.
TEST_TIM	R*4	Average test execution time in seconds	See TSR Statistics.
TEST_MIN	R*4	Lowest test result value	See TSR Statistics.
TEST_MAX	R*4	Highest test result value	See TSR Statistics.
TST_SUMS	R*4	Sum of test result values	See TSR Statistics.
TST_SQRS	R*4	Sum of squares of test result values	See TSR Statistics.

Test Name

For the TEST_NAM field, IG-XL determines the test name in this order:

•	If TheExec.Flow.TestLimit specifies a TName for a limits check, that name is used.

- If TestLimit does not specify a TName, the TName column from the Flow Table sheet for this test instance is used.

- If the Flow Table sheet does not specify a TName, the test instance name is used.

The TEST_NAM field contains this test name plus additional information, depending on whether the test results are datalogged as FTRs, MPRs, or PTRs.
For a functional test, the TEST_NAM is the test name plus the pattern, as specified for the test instance. The pattern can be a pattern file, set, or group, as supported by the tester. For example: FVEven_MVOdd PE_CPU_MV.pat
For a parametric test that is datalogged as an MPR, the TEST_NAM field contains the test name, as determined by the steps given above. There is no additional information.
For a parametric test that is datalogged as a PTR, there are more options. The default TEST_NAM format is:
test-name pin-name channel-number
For example: FVMI vcc 19.x310
Using DataCollect Setup (the Display Options window of the Setups tab) or VBT (properties under TheExec.Datalog.Setup.DatalogSetup), you can specify a nondefault format for a parametric test, as follows.
You can remove the channel number from the TEST_NAM:

- DataCollect Setup: Disable Channel Number in TSR Record check box (this is enabled only if the Test Synopsis Data box is checked).

- VBT: ChannelNo (this property is meaningful only if Synopsis is set to True, which is the default).

You can use the test name as it is specified in the PTR TEST_TXT field:

- DataCollect Setup: TSR test_nam same as PTR test_txt.

- VBT: TSRTestNameSameAsPTRTestTxt.

You can also modify the format of the PTR TEST_TXT field. If you select this option for the TSR, the TSR TEST_NAM field format will be whatever the currently specified PTR TEST_TXT format is. For more information, see TEST_TXT under the description of the PTR.
“Sequencer” Name

Because IG-XL does not support the concept of a sequencer, the SEQ_NAME field is used for either the test instance name or the Flow Table sheet name. The default is the test instance name. Because the test name is already recorded in the TEST_NAM field, you can specify that SEQ_NAME should have the Flow Table sheet name instead:

- DataCollect Setup: on the Setups tab, on the Display Options window, check Report By Flow Sheet Name in TSR Record.

- VBT: Set TheExec.Datalog.Setup.DatalogSetup.TSR_ReportByFlowSheetName to True.

TEST_LBL

By default, IG-XL does not write valid values to this fields. You can specify a value to be written using:

Call TheExec.Datalog.SetTSRTestLabel(TestLbl)

For more information, see SetTSRLabel in the VBT Syntax Reference.

Note that the string you specify is written to all TSRs for all tests.

DataCollect Setup has no control to specify this value.

TSR Statistics

By default, IG-XL does not write valid values to the TST_TIM, TST_MIN, TEST_MAX, TST_SUMS, and TST_SQRS fields. You can specify a value to be written to these fields using:

TheExec.Datalog.setup.DatalogSetup.EnableTSRStatistics(bFillTSRStats)

For more information, see EnableTSRStatistics in the VBT Syntax Reference.

If the method is called with the argument True, IG-XL computes the statistics and writes to these fields.

In addition, it sets the appropriate bits of the OPT_FLAG field to indicate that these statistic fields have valid values.

DataCollect Setup has no control to specify these values.

StdfIgxlRecords.2.20

< >

IG-XL Support for STDF Record Types : Parametric Test Record (PTR)

Parametric Test Record (PTR)
STDF References: Parametric Test Record (PTR)
Contains the results of a single execution of a parametric test in the test program. The first occurrence of this record also establishes the default values for all semistatic information about the test, such as limits, units, and scaling.
One PTR is written for each execution of TheExec.Flow.TestLimit in the program. For the PPMU, the statement TheHdw.PPMU(Pins).Test writes a PTR. If writing MPRs is enabled, MPRs may be written instead of PTRs. See How IG-XL Groups Results into MPRs in the MPR section.
In addition, you can use these VBT statements to write a PTR: TheExec.Datalog.WriteParametricResult and WriteParametricResultOptLoHi. When the following description refers to a parameter of TestLimit, you can assume that the comparable parameters of the two Datalog statements are meant as well.

PTR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
TEST_NUM	U*4	Test number	The TNum number for this test in the Flow Table sheet. If TNum is not specified, increment the previous test number by 1. If no TNum values are specified, the first test starts at 0.
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.

SITE_NUM	U*1	Test site number	The site where this test was executed.
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	Eight bits indicating the status of the test result. Individual bit positions have the following meanings: • 0: test passed • 0x1: alarm detected during testing • 0x2: value in the RESULT field is invalid • 0x4: test result is unreliable • 0x8: timeout occurred • 0x10: test was not executed • 0x20: test aborted • 0x40: test completed with no pass/fail indication • 0x80: test failed
PARM_FLG	B*1	Parametric test flags (drift, etc.)	Eight bits giving information about the parametric test. Individual bit positions have the following meanings: • 0: no special information • 0x1: scale error • 0x2: drift error

			• 0x4: oscillation detected • 0x8: measured value is higher than test limit • 0x10: measured value is lower than test limit • 0x20: test passed alternate limits (0 = failed or passed standard limits) • 0x40: if result = low limit, then result is pass (0 = result is fail) • 0x80: if result = high limit, then result is pass (0 = result is fail)
RESULT	R*4	Test result	The test result value. This value come from the resultVal parameter of TheExec.Flow.TestLimit.
TEST_TXT	C*n	Test description text or label	Test name plus other information, configurable by the user. See TEST_TXT.
ALARM_ID	C*n		Not written.
OPT_FLAG	B*1	Optional data flag	Indication whether other fields are valid. Individual bit positions have this meaning: • 0x01: result value is invalid • 0x02: reserved - must be 1 • 0x04: no low spec limit - always 1, because IG-XL does not write this value

			• 0x08: no high spec limit - always 1, because IG-XL does not write this value • 0x10: low limit is invalid • 0x20: high limit is invalid • 0x40: no low limit for this test • 0x80: no high limit for this test The value may be a combination of these bits.
RES_SCAL	I*1	Test results scaling exponent	Valid values are: • 15: femto (1e-15) • 12: pico (1e-12) • 9: nano (1e-9) • 6: micro (1e-6) • 3: milli (1e-3) • 2: percent (1e-2) • 0: none (1e-0) • -3: Kilo (1e+3) • -6: Mega (1e+6)

			• -9: Giga (1e+9) • -12: Tera (1e+12) The value is taken from the scaletype parameter of TheExec.Flow.TestLimit or from the Scale column of the Flow Table sheet.
LLM_SCAL	I*1	Low limit scaling exponent	The same value as RES_SCAL.
HLM_SCAL	I*1	High limit scaling exponent	The same value as RES_SCAL.
LO_LIMIT	R*4	Low test limit value	The value is taken from the lowVal parameter of TheExec.Flow.TestLimit or from the LoLim column of the Flow Table sheet.
HI_LIMIT	R*4	High test limit value	The value is taken from the hiVal parameter of TheExec.Flow.TestLimit or from the HiLim column of the Flow Table sheet.
UNITS	C*n	Test units	Test units. Values written include: • dB (decibels) • A (ampere) • hz (Hertz) • V (volt) • s (seconds) The value is based on the unit parameter of TheExec.Flow.TestLimit or the Units column of the Flow Table sheet. If TestLimit specifies a custom unit, the customUnit value is used.

			The Units column supports any custom-defined unit.
C_RESFMT	C*n	ANSI C result format string	The ANSI C format string to be used for outputting the result. For example, "%7.2f". The value is taken from the formatStr parameter of TheExec.Flow.TestLimit or from the Format column of the Flow Table sheet. The default format is" %f ".
C_LLMFMT	C*n	ANSI C low limit format string	The same value as C_RESFMT.
C_HLMFMT	C*n	ANSI C high limit format string	The same value as C_RESFMT.
LO_SPEC	R*4		Not written (OPT_FLAG bit 2 = 1)
HI_SPEC	R*4		Not written (OPT_FLAG bit 3 = 1)

Default Data Written Only to First PTR

The PTR fields for specifying high and low limits and for specifying output formatting (that is, all fields following the OPT_FLAG) are usually the same for all PTRs with the same test number. To save space, this data is written only for the first PTR. Subsequent PTRs with the same test number have the missing/invalid data value (this is usually a bit setting in OPT_FLAG), indicating that the value has been set in the first PTR. If a subsequent PTR has a valid value for one of these fields, that value is used for that one record only; PTRs after that one use the default established by the first PTR.

Note that these fields are written to the first PTR only if the datalog setup format is Expanded. If the Short format is selected, these fields are not written, even to the first PTR. For more information, see Built-In Setups (Short and Expanded) in the DataCollect Setup section.

TEST_TXT

IG-XL uses the TEST_TXT field to record information about the test. The format depends on the datalog setup format selected. On selecting formats, see Built-In Setups (Short and Expanded) in the DataCollect Setup section.

If the Short datalog setup format is selected, the format is this:

test-instanceName <> TName

That is, the name of the test instance from the Parameter column and the test name from the TName column of the Flow Table sheet.

For example:

FVMI <> FVMI12

where “FVMI” is the name of the test instance and “FVMI12” is the name specified by TName.
If the Expanded datalog setup format is selected, you can control how this information is to appear. The default format for Expanded datalog is:
test-instanceName pinName channelNumber <> string
where “string” is one of the following:

- The test name, as specified by the TName column of the Flow Table sheet, in the case where the PTR is generated by the execution of TheExec.Flow.TestLimit.

- The comments parameter of TheExec.Datalog.WriteParametricResult or .WriteParametricResultOptLoHi.

For example:
FVMI12 vcc 19.x304 <> FVMI12
You can specify different formats. Each option is available from DataCollect Setup from the Setups tab, which brings up the Display Options window. This window has a section labelled PTR test_txt Options. The options from this section are also available in VBT under TheExec.Datalog.Setup.DatalogSetup. The following descriptions mention both the control on the Display Options window and the VBT statement.

- Use IMAGE-style format.

This is the order used by Teradyne’s IMAGE tester executive:
test-instanceName pinName channelNumber <> string
For example: FVMI12 <> FVMI12 vcc 19.x304
Display Options: Output PTR test_txt in IMAGE format check box
VBT: ReversePTRTestTxt

- Disable the instance name:

Do not include the test instance name in the text:
pinName channelNumber <> string
For example: vcc 19.x304 <> FVMI12
Display Options: Disable Instance Name check box
VBT: DisableInstanceNameInPTR
If this check box is not checked, the two radio buttons below it are enabled. You can choose these options:

Test Instance Name	Use the name of the test instance as specified in the Parameter column of the Flow Table sheet. For example: FVMI <> FVMI12 vcc 19.x304 (where “FVMI” is the name of the test instance and “FVMI12” is the name specified by TName; note that this option does not apply to the contents of the “string” or comment)
Flow TName or Test Instance Name	Use the test name from the Flow, if one is specified in the TName column of the Flow Table. If no TName is specified, use the test instance name.

In VBT, use the statement PTR_InstanceNameIsTINameOnly. Setting it to True is the same as selecting Test Instance Name; setting it to False is the same as selecting Flow TName or Test Instance Name.

- Disable the channel number.

Do not include the channel number in the text:
test-instanceName pinName <> string
Display Options: Disable Channel Number check box
VBT: DisableChannelNumberInPTR

- Disable the pin name.

Do not include the in name in the text:
test-instanceName channelNumber <> string
Display Options: Disable Pin Name check box
VBT: DisablePinNameInPTR

StdfIgxlRecords.2.21

< >

IG-XL Support for STDF Record Types : Multiple-Result Parametric Record (MPR)

Multiple-Result Parametric Record (MPR)
STDF Reference: Multiple-Result Parametric Record (MPR)
Contains the results of a single execution of a parametric test in the test program where that test returns multiple values. The first occurrence of this record also establishes the default values for all semistatic information about the test, such as limits, units, and scaling. The MPR is related to the Test Synopsis Record (TSR) by test number, head number, and site number. See Test Synopsis Record (TSR).
If writing MPRs is enabled (see Enabling MPR Creation), IG-XL creates an MPR in these general circumstances:

- When a test produces multiple results per pin. The results of such a test can be stored in one MPR rather than multiple PTRs, one per result.

- When a parametric test is run on a pin group with separate results for each pin. The results for the entire pin group can be stored in one MPR rather than multiple PTRs, one per pin.

For more information, see How IG-XL Groups Results into MPRs.

MPR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
TEST_NUM	U*4	Test number	See TEST_NUM Field.

HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_NUM	U*1	Test site number	The site where this test was executed.
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	Eight bits indicating the status of the test result. Individual bit positions have the following meanings: • 0: test passed • 0x1: alarm detected during testing • 0x2: not supported - this bit must be 0 • 0x4: test result is unreliable • 0x8: timeout occurred • 0x10: test was not executed • 0x20: test aborted • 0x40: test completed with no pass/fail indication • 0x80: test failed The status is taken for each pin in the pin group. The results are then OR'ed together to give the value written to this field.
PARM_FLG	B*1	Parametric test flags (drift, etc.)	Eight bits giving information about the parametric test. Individual bit positions have the following meanings: • 0: no special information

			• 0x1: scale error • 0x2: drift error • 0x4: oscillation detected • 0x8: measured value is higher than test limit • 0x10: measured value is lower than test limit • 0x20: test passed alternate limits (0 = failed or passed standard limits) • 0x40: if result = low limit, then result is pass (0 = result is fail) • 0x80: if result = high limit, then result is pass (0 = result is fail) The status is taken for each pin in the pin group. The results are then OR'ed together to give the value written to this field.
RTN_ICNT	U*2	Count (j) of PMR indexes	The number of items in the arrays of PMR indexes (RTN_INDX) and returned states (RTN_STAT). For IG-XL, RTN_ICNT is always the same as RSLT_CNT. See Arrays of Pins, Results, and Returned States.
RSLT_CNT	U*2	Count (k) of returned results	The number of items in the array of returned results (RTN_RSLT). For IG-XL, RTN_ICNT is always the same as RSLT_CNT. See Arrays of Pins, Results, and Returned States.

RTN_STAT	jxN*1	Array of returned states	Currently, this information is not available for datalogging, and the returned state is always set to 4, which means “undetermined.” See Arrays of Pins, Results, and Returned States.
RTN_RSLT	kxR*4	Array of returned results	The measured results of the parametric test. See Arrays of Pins, Results, and Returned States.
TEST_TXT	C*n	Descriptive text or label	The test name taken from the TName column of the Flow Table sheet. If no TName is given, the name of the test instance in the Command Parameter column is used.
ALARM_ID	C*n	Name of alarm	Not written.
OPT_FLAG	B*1	Optional data flag	Indication whether other fields are valid. Individual bit positions have this meaning: • 0x01: result value is invalid • 0x02: START_IN and INCR_IN are invalid-always 1, because IG-XL does not write these values • 0x04: no low spec limit - always 1, because IG-XL does not write this value • 0x08: no high spec limit - always 1, because IG-XL does not write this value • 0x10: low limit is invalid • 0x20: high limit is invalid • 0x40: no low limit for this test

			• 0x80: no high limit for this test The value may be a combination of these bits.
RES_SCAL	I*1	Test result scaling exponent	Valid values are: • 15: femto (1e-15) • 12: pico (1e-12) • 9: nano (1e-9) • 6: micro (1e-6) • 3: milli (1e-3) • 2: percent (1e-2) • 0: none (1e-0) • -3: Kilo (1e+3) • -6: Mega (1e+6) • -9: Giga (1e+9) • -12: Tera (1e+12) The value is taken from the scaletype parameter of TheExec.Flow.TestLimit or from the Scale column of the Flow Table sheet.
LLM_SCAL	I*1	Test low limit scaling exponent	The same value as RES_SCAL.
HLM_SCAL	I*1	Test high limit scaling exponent	The same value as RES_SCAL

LO_LIMIT	R*4	Test low limit value	The value is taken from the lowVal parameter of TheExec.Flow.TestLimit or from the LoLim column of the Flow Table sheet.
HI_LIMIT	R*4	Test high limit value	The value is taken from the hiVal parameter of TheExec.Flow.TestLimit or from the HiLim column of the Flow Table sheet.
START_IN	R*4	Starting input value (condition)	Not written.
INCR_IN	R*4	Increment of input condition	Not written.
RTN_INDX	jxU*2	Array of PMR indexes	The PMR index numbers for the pins whose results are contained in this MPR. See Arrays of Pins, Results, and Returned States.
UNITS	C*n	Units of returned results	Test units. Values written include: • dB (decibels) • A (ampere) • hz (Hertz) • V (volt) • s (seconds)
UNITS_IN	C*n	Input condition units	Not written.
C_RESFMT	C*n	ANSI C result format string	The ANSI C format string to be used for outputting the result. For example, "%7.2f". The value is taken from the formatStr parameter of TheExec.Flow.TestLimit or from the Format column of the Flow Table sheet. The default format is" %f ".

C_LLMFMT	C*n	ANSI C low limit format string	The same value as C_RESFMT.
C_HLMFMT	C*n	ANSI C high limit format string	The same value as C_RESFMT.
LO_SPEC	R*4	Low specification limit value	Not written.
HI_SPEC	R*4	High specification limit value	Not written.

Enabling MPR Creation

By default, IG-XL writes an individual PTR for each parametric test result. To have IG-XL group the appropriate parametric results into MPRs, you must explicitly enable writing of MPRs using the VBT method TheExec.Datalog.Setup.DatalogSetup.EnableMultipleParametricResult. Call this method from the exec interpose function OnProgramLoaded. DataCollect Setup has no control for enabling MPRs.

Once writing MPRs is enabled, IG-XL automatically groups the appropriate results and writes them as MPRs. See the next topic. If parametric results do not meet the MPR criteria, they are written as PTRs.

How IG-XL Groups Results into MPRs

IG-XL can group parametric results into an MPR if the test conditions are uniform across the results. If these conditions are uniform at the time that the parametric results are to be datalogged, IG-XL writes a single MPR rather than multiple PTRs.

The test conditions that IG-XL considers are different, depending on whether short or expanded datalog format is selected. In expanded format, the test conditions are the low and high limit, the measured unit, and the force value and unit. In short format, only the measured unit is datalogged, and it is the only criterion for grouping parametric results into an MPR. Therefore, parametric results that are grouped into an MPR in short format may be datalogged as separate PTRs in expanded format.

The following VBT calls can generate MPRs:

- TheExec.Flow.TestLimit
- TheHdw.PPMU.Pins(pin-list).Test
- TheHdw.DCIO.Pins(pin-list).MRAM.Test

The following sections cover each of the calls.

TestLimit Statement

The VBT statement TheExec.Flow.TestLimit compares test results to limits and datalogs the results. If the result value is a PinListData (PLD) object, TestLimit generates one MPR for each site in the PLD. It is assumed that the results stored in a PLD object were created under uniform test conditions. TestLimit generates MPRs from a PLD whether the datalog format is short or expanded.

Note that TestLimit does not generate an MPR if the result value is a PinData object, that is, data for a single pin. A PinData object generates one PTR per site.

In addition to the PLD, TestLimit generates MPRs if the result value is a ParametricResult object. If the result value is an array of ParametricResult objects, an MPR is generated for each element in the array.

PPMU Test Statement

Note that this section describes VBT code that is written to perform a PPMU measurement. If you are using the Teradyne-supplied PinPMU_T library test, the code handles the grouping and writing of MPRs. The following is relevant only if you are writing code for a PPMU test.

The VBT statement TheHdw.PPMU.Pins(pin-list).Test performs a PPMU measurement for each pin, compares the measurements to limits, and datalogs the result. If Test specifies multiple pins, and if the test conditions are uniform, it generates one MPR per site.

The following examples illustrate when the PPMU Test statement creates MPRs. For simplicity, the examples assume a single site.

In the simplest case, a test on multiple pins generates a single MPR, because the single ForceV statement ensures that the test conditions are uniform:

```
Call TheHdw.PPMU(p1,p2).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p1,p2).Test
```

In the next case, the test conditions are the same for the sets of pins in both ForceV statements, and a single MPR is generated:

```
Call TheHdw.PPMU(p1,p2).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p3,p4).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p1,p2,p3,p4).Test
```

In the next case, even though the test conditions are uniform for both sets of pins, each Test statement generates an MPR, resulting in two MPRs:

```
Call TheHdw.PPMU(p1,p2).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p1,p2).Test
Call TheHdw.PPMU(p3,p4).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p3,p4).Test
```

In the next case, the ForceV statements are different for the two sets of pins:

```
Call TheHdw.PPMU(p1,p2).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p3,p4).ForceV(Volt2, MeasCurrRange2, HiLim2,LoLim2)
Call TheHdw.PPMU(p1,p2,p3,p4).Test
```

If the Expanded datalog format is selected, the test conditions are not uniform, and four PTRs are generated. If the Short format is selected, only the measured unit is used as a test condition. Because both ForceV statements have the same measure unit, the test conditions for both sets of pins are uniform, and one MPR is generated.

In the next case, the test conditions are set by ForceV and ForceI statements:

```
Call TheHdw.PPMU(p1,p2).ForceV(Volt1, MeasCurrRange1, HiLim1,LoLim1)
Call TheHdw.PPMU(p3,p4).ForceI(Curr2, ForceCurrRange2, HiLim2,LoLim2)
Call TheHdw.PPMU(p1,p2,p3,p4).Test
```

Because the measured units are different, the test conditions are not uniform, even for Short datalog format. Four PTRs are generated in either Expanded or Short datalog format.

Note that serial testing needs special attention, because you need to indicate the nesting of results. You do this using the VBT Datalog statements StartMultipleParametricResult and WriteMultipleParametricResult.

For more information, see StartMultipleParametricResult.

DCIO Test Statement

The DCIO Test statement is similar to the PPMU Test statement. A test on multiple pins generates one MPR if the test conditions are identical for all pins. In Short datalog format, the only test condition is the measurement units.

The following example show how DCIO results are grouped in MPRs.

In the first example, test conditions are the same for all pins, so one MPR is generated:

```
TheHdw.DCIO.Pins("ALL_DCIO").Mode = tlDCIOModeFVMI
TheHdw.DCIO.Pins("ALL_DCIO").StartState = tlDCIOStartStateForceV
TheHdw.DCIO.Pins("ALL_DCIO").Levels.Value(tlDCIOLevelPMUFV) = ForceVal
TheHdw.DCIO.Pins("ALL_DCIO ").MRAM.Test tlDCIOMRAMTestAll
```

In the second example, different pins have different test conditions:

```
TheHdw.DCIO.Pins("ALL_DCIO").Mode = tlDCIOModeFVMI
TheHdw.DCIO.Pins("ALL_DCIO").StartState = tlDCIOStartStateForceV
TheHdw.DCIO.Pins("DCIO_0,DCIO_1").Levels.Value(tlDCIOLevelPMUFV) = 0
TheHdw.DCIO.Pins("DCIO_2, DCIO_3").Levels.Value(tlDCIOLevelPMUFV) = 2.5
TheHdw.DCIO.Pins("ALL_DCIO ").MRAM.test tlDCIOMRAMTestAll
```

In Expanded datalog format, the test conditions are not the same, and four PTRs are generated. In Short format, only the measured units must match, so one MPR is generated.

TEST_NUM Field

The TEST_NUM field contains the TNum number for this test in the Flow Table sheet. If the sheet does not specify a TNum, IG-XL generates a test number. Each MPR has the test number for the first pin in its pin group or list. A test number is generated internally for each subsequent pin in the group, but this number is not written to the file. The next MPR then increments from the most recent internally generated test number.

For example, assume three tests and limits checks, on three pins, two pins, and two more pins. The test numbers for the three resulting MPRs are generated as follows:

For (p0,p1,p2):

p0 = Test Number 0

p1 = Test Number 1

p2 = Test Number 2

MPR TEST_NUM = 0 (from the first pin in this group)

For (p3,p4):

p3 = Test Number 3 (incremented from the last pin of the previous group)

p4 = Test Number 4

MPR TEST_NUM = 3 (from the first pin in this group)

For (p5,p6):

p5 = Test Number 5 (incremented from the last pin of the previous group)

p6 = Test Number 6

MPR TEST_NUM = 5 (from the first pin in this group)

Effect of Changed Test Numbers

Note that an existing program that is now run with MPRs enabled may have different test numbers. For example, if a program's PPMU test has 10 pins, the result before may be 10 PTRs with test numbers from 1 through 10. If the program is now run with MPRs enabled, the result may be one PTR with 10 entries, all under test number 1. This may confuse any data analysis software that tries to compare earlier with current results. To maintain consistency with historical data, you can give the MPR the test number 11, so that the historical data for tests 1 through 10 will still be valid.

Arrays of Pins, Results, and Returned States

The MPR uses arrays to return the multiple pins (RTN_INDX), results (RTN_RSLT), and returned states (RTN_STAT). The arrays all have the same size, given by RTN_ICNT (count of PMR indexes and count of returned states) and RSLT_CNT (count of returned results), which are always equal. There is one array element for each result, so that the nth element of the arrays indicates the result value (RTN_RSLT(n)) and returned state (RTN_STAT(n)) for the pin in the PMR index indicated by the number at RTN_INDX(n).

The following examples show how the arrays are structured. The examples show only the pin (RTN_INDX) and result (RTN_RSLT), because the returned state (RTN_STAT) is comparable to RTN_RSLT. In each example, pins p1, p2, and p3 are assumed to correspond to PMR index numbers 1, 2, and 3.

Example 1:

A test on pins p1, p2, and p3 produces one result per pin.

The arrays have 3 elements:

RTN_INDX	1.2	1.5	0.9
RTN_RSLT	1	2	3

Example 2:

A test on pins p1, p2, and p3 produces two results per pin. The mode is “Each sample.”

The arrays have 6 elements, with 2 results for each of 3 pins. The results for each pin are stored consecutively:

RTN_INDX	1.2	1.5	0.9	1.3	1.0	1.4
RTN_RSLT	1	1	2	2	3	3

Example 3:

A test on pins p1, p2, and p3 produces two results per pin. The mode is “Average.”

The arrays have 3 elements, with 1 average of the 2 results for each pin:

RTN_INDX	1.4	1.1	0.8
RTN_RSLT	1	2	3

Default Data Written Only to First MPR

The MPR fields for specifying high and low limits and for specifying output formatting (that is, all fields following the OPT_FLAG) are usually the same for all MPRs with the same test number. To save space, this data (except for RTN_INDX) is written only for the first MPR. Subsequent MPRs with the same test number have the missing/invalid data value (this is usually a bit setting in OPT_FLAG), indicating that the value has been set in the first MPR. If a subsequent MPR has a valid value for one of these fields, that value is used for that one record only; MPRs after that one use the default established by the first MPR.

Note that these fields are written to the first MPR only if the datalog setup format is Expanded. If the Short format is selected, these fields are not written, even to the first MPR.

For more information, see Built-In Setups (Short and Expanded) in the DataCollect Setup section.

< >

StdfIgxlRecords.2.22

< >

IG-XL Support for STDF Record Types : Functional Test Record (FTR)

Functional Test Record (FTR)

STDF Reference: Functional Test Record (FTR)

Contains the results of the single execution of a functional test in the test program.

Note that the fields that are written depend on the datalog setup that you have selected. See Effect of Datalog Setups.

FTR Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
TEST_NUM	U*4	Test number	The TNum number for this test in the Flow Table sheet. If TNum is not specified, increment the previous test number by 1. If no TNum values are specified, the first test starts at 0.
HEAD_NUM	U*1	Test head number	Always set to 1. IG-XL does not support multiple test heads.
SITE_NUM	U*1	Test site number	The site where this test was executed.
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	Eight bits indicating the status of the test result. The individual bit positions have the following meanings: • 0: test passed

			• 0x1: alarm detected during testing • 0x2: reserved - must be 0 • 0x4: test result is unreliable • 0x8: timeout occurred • 0x10: test was not executed • 0x20: test aborted • 0x40: test completed with no pass/fail indication • 0x80: test failed
OPT_FLAG	B*1	Optional data flag	Indication whether other fields are valid. Individual bit positions have this meaning: • 0x01: CYCL_CNT data is invalid • 0x02: REL_VADR data is invalid • 0x04: REPT_CNT data is invalid • 0x08: NUM_FAIL data is invalid • 0x10: XFAIL_AD and YFAIL_AD data are invalid. Because IG-XL does not write these fields, this bit is always 1. • 0x20: VECT_OFF data is invalid. Because IG-XL does not write this field, this bit is

			always 1. • 0x40: reserved - must be 1 • 0x80: reserved - must be 1 The value is a combination of these bits.
CYCL_CNT	U*4	Cycle count of vector	The cycle count of the failing vector.
REL_VADR	U*4	Relative vector address	The address of the failing vector relative to the start of the pattern.
REPT_CNT	U*4	Repeat count of vector	
NUM_FAIL	U*4	Number of pins with 1 or more failures	Number of pins with failures.
XFAIL_AD	I*4		Not written.
YFAIL_AD	I*4		Not written.
VECT_OFF	I*2	Offset from vector of interest	See VECT_OFF below.
RTN_ICNT	U*2	Count (j) of return data PMR indexes	
PGM_ICNT	U*2	Count (k) of programmed state indexes	
RTN_INDX	jxU*2	Array of return data PMR indexes	
RTN_STAT	jxN*1	Array of returned states	An array of returned states, one for each of the pins in the RTN_INDX array. The valid returned states are defined in the RTN_CHAR field of the PLR. The number in this array corresponds to the position of the character in the PLR field. The numbers refer to these return states:

			• 0: L • 1: H • 2: M • 3: G • 4: * (UltraFLEX only) • F: The returned value is not one of these valid values.
PGM_INDX	kxU*2	Array of programmed state indexes	
PGM_STAT	kxN*1	Array of programmed states	An array of programmed states, one for each of the pins in the PGM_INDX array. The valid returned states are defined in the PGM_CHAR field of the PLR. The number in this array corresponds to the position of the character in the PLR field. The numbers refer to these programmed states: • 0: 0 • 1: 1 • 2: L • 3: H • 4: M

			• 5: V • 6: X • 7: W • A: * (UltraFLEX only) • F: The programmed value is not one of these valid values.
FAIL_PIN	D*n	Failing pin bitfield	A variable-length bit-encoded field. Encoded with PMR index 0 in bit 0 of the field, PMR index 1 in the 1st position, and so on. Bits representing PMR indexes of failing pins are set to 1.
VECT_NAM	C*n	Vector module pattern name	The pattern module name or pattern file name, whichever was specified as the parameter to the VBT statement.
TIME_SET	C*n	Time set name	The time set that the pattern specifies for the vector. This value is written only if a custom setup file is used with the Timing Sets box checked. See Functional Setup Dialog Box.
OP_CODE	C*n	Vector Op Code	Any opcode that is on the vector in the pattern. This value is written only if a custom setup file is used with the Opcodes box checked. See Functional Setup Dialog Box.
TEST_TXT	C*n	Descriptive text or label	IG-XL uses this field to write the test name. The name is taken from the TName column on the Flow Table sheet. If there is no TName value, the test instance name is used.
ALARM_ID	C*n		Not written.

PROG_TXT	C*n	Not written.
RSLT_TXT	C*n	Not written.
PATG_NUM	U*1	Not written. 255: missing/invalid data
SPIN_MAP	D*n	Not written.

Effect of Datalog Setups

The datalog setup format that you select affects what fields are filled in. See Built-In Setups (Short and Expanded) in the DataCollect Setup section.

If you select the Short format, the following fields are filled in: TEST_NUM, HEAD_NUM, SITE_NUM, TEST_FLG, VECT_NAM, and TEST_TXT. OPT_FLAG is set to 0xff, meaning that the other fields are not written.

If you select Expanded format, all fields are written as described above.

If you use a custom setup with expanded format, you can specify that the TIME_SET and OP_CODE fields are to be written. The other options on the dialog box do not affect whether STDF fields are written. See Functional Setup Dialog Box.

VECT_OFF

By default, the VECT_OFF value is not written. To enable it, set TheExec.Datalog.Setup.DatalogSetup.EnableFTRVectorOffset to True. See EnableFTRVectorOffset in the VBT Syntax Reference.

For the definition of this field, see Functional Test Record (FTR) in the STDF Specification. Briefly, it is the integer offset from the vector.

You indicate the vector of interest by specifying the HRAM Trigger option. If writing VECT_OFF is enabled, the datalog service collects all trigger information, such as trigger type, cycle information, pattern name, and label, and sends it to the STDF data stream. The VECT_OFF value is calculated based on this trigger information and written to the FTR.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.23

< >

IG-XL Support for STDF Record Types : Begin Program Section Record (BPS)

Begin Program Section Record (BPS)
STDF Reference: Begin Program Section Record (BPS)
Marks the beginning of a run of the job.
One BPS is written for each run of the job.

BPS Data Fields

Field Name	Data Type	Field Description	Value Written By IG-XL
SEQ_NAME	C*n	Program name	The name of the Flow Table sheet.

BPS/EPSs and Sampling
The number of BPS/EPS pairs written depends on whether sample datalogging is being used. If there is no sampling and every result is being datalogged, then a BPS/EPS pair is written for each part.
If sample datalogging is being used, for a sample rate of n, every nth part is datalogged. By default, a BPS/EPS pair for a test run is written only if parts are datalogged for that run.
You can ensure that a BPS/EPS pair is written for each test run using one of these methods:

- In DataCollect Setup, on the Setups tab Display Options window, check the box labelled Full Sample STDF Part Records.

- In VBT, set this property to True:
TheExec.Datalog.setup.DatalogSetup.FullSampleStdfPartRecords

If this option is enabled, the default behavior is overridden. A BPS/EPS pair is written for each test run, even if no parts are datalogged in the test run. In addition, a PIR/PRR pair is written for every part, whether or not the part is datalogged.

< >

2015 Teradyne, Inc. All rights reserved.

StdfIgxlRecords.2.24

< >

IG-XL Support for STDF Record Types : End Program Section Record (EPS)

End Program Section Record (EPS)
STDF Reference: End Program Section Record (EPS)
Marks the end of a run of the job.
One EPS is written for each run of the job.
Apart from the standard header fields, an EPS has no data fields. BPS/EPS pairs can be nested. Each EPS matches the last unmatched BPS.

< >

StdfIgxlRecords.2.25

< >

IG-XL Support for STDF Record Types : Generic Data Record (GDR)

Generic Data Record (GDR)
STDF Reference: Generic Data Record (GDR)
IG-XL supports a GenericDataRecord VBT object lets you write GDRs to the STDF file. For more information, see GenericDataRecord in the VBT Syntax Reference.
IG-XL datalogging uses GDRs when the site count is greater than 255. In this case, GDR contains two data fields: the ASCII string EXTENDEDSITES and the actual site count. For more information, see Extended Site Numbers and Counts.
Other IG-XL tools and utilities can use GDRs to write specialized information to the datalog stream. For example, see Test Audit Output in the section on the Real Time Audit Tool.

< >

StdfIgxlRecords.2.26

< >

IG-XL Support for STDF Record Types : Datalog Text Record (DTR)

Datalog Text Record (DTR)
STDF Reference: Datalog Text Record (DTR)
Contains text information that is to be included in the datalog printout.
The DTR field TEXT_DAT is a string contains the text to be datalogged.
IG-XL writes a DTR in the following circumstances:

- If a test fails to execute, one DTR record is written to the STDF file with the cause of the failure.
- The execution of an error-print or logprint opcode in the Flow Table sheet writes a DTR with the message that is the parameter to the opcode.
- The VBT statement TheExec.Datalog.WriteComment writes a DTR to the STDF file.

You can disable the writing of DTRs in one of these ways:

- In DataCollect Setup, the Setups tab has the check box Disable Datalog Text Record.
- In VBT, use the statement TheExec.Datalog.Setup.DatalogSetup.DisableDtr.

< >